

GRACE

Review of Palak's working design for the editorial tracker

Prepared by: Stephen Gadd · **Date:** 25 August 2026 · **For:** Ruth Mostern, Palak Vashist, Alexandra Straub · **Purpose:** response to "Tracking people, projects, and datasets — a working design for our editorial workflow"

Palak's draft describes a **coherent system**. The entity graph in the prose and the diagram agree with each other, the cardinalities are stated and correct, and the "how is it controlled / what values are allowed" discipline that Ruth asked for has produced exactly the data dictionary a build needs. This note proposes **four structural fixes**, answers the five questions Palak left open, and raises **one governance point** that has not yet been put on the table.

The system is GRACE — *Gazetteer Register And Contact Engagement* — a Django app inside WHG, sitting alongside `datasets`, `collection` and the rest. The name says what it holds: a register of the gazetteers, people, projects and sources we track, and a record of our dealings with them.

A note on terms. Every contribution to WHG is a **gazetteer**, the single term replacing the older "dataset" / "collection" split. Today a gazetteer is realised in the software as a *Dataset* or a *Collection*; the concept extends to routes, itineraries and networks. A *print historical gazetteer* is a **source** we might turn *into* a WHG gazetteer — a lead, not a gazetteer in the platform sense. §7 explains why the draft's persistent "dataset" should not simply be find-and-replaced, and what to do instead.

1. What is working, and should not be touched

- **"Registers, not phases."** **KEEP** Work does not move in a line — a gazetteer can be in live correspondence with its owner *and* halfway through review. Naming these registers, in the old sense of a running official list, buys us out of a false sequence.
- **Two questions per field.** **KEEP** "How is it controlled" and "what values may it hold", answered for every field, is the difference between a spreadsheet and a system. The four-way classification maps cleanly onto Django: a pick-list becomes a lookup table, a link becomes a `ForeignKey`, a date is a date, and free text is confined to where consistency genuinely does not matter.
- **An engagement is with a person, not with a record.** **KEEP** The single best modelling decision in the draft. It is what makes "open one person and see every conversation" possible, and it is why the register survives a project being renamed or a gazetteer being withdrawn.
- **Action items as real tasks.** **KEEP** Palak's own correction — from a free-text plan to an owned, dated task — is right, and it is what makes the reminder rule work at all.
- **Print gazetteers folded into Sources.** **KEEP** Giving them their own record would duplicate the bibliography for no gain. One source type is simpler and correct.

2. Fix 1 — point at the Gazetteer Register rather than restating it

The proposed *Dataset tracking status* — `on our radar`; `in internal review`; `revisions requested`; `ready to publish`; `in external peer review`; `published`; `indexed`; `not indexed`; `on hold`; `declined` — conflates two axes. Some of those values are editorial judgements. Others are machine facts, and Palak

flags the difficulty herself: *"the dashboard stages are guesses... our technical director will know how they line up"*.

They line up onto something that already exists. The **Gazetteer Register** (`GazetteerRegistryEntry`) is WHG's canonical list of gazetteers, kept current by an idempotent push from the indexing side on every successful ingest. It is already the unified list this design wants: `entry_class` spans `authority` and `dataset`, so external authorities and WHG contributions sit in one table, and `gazetteer_type` already carries `standard / itinerary / network` — the same widening of "gazetteer" the terminology note describes.

So GRACE should not restate any of that. Its gazetteer record carries a **nullable foreign key to the Register** and reads through it:

```
class TrackedGazetteer(models.Model):
    # GRACE's own record. Exists from the moment something lands on our
    # radar – long before there is anything in WHG to point at.
    title = models.CharField(...) # what we called it when we first heard of it
    registry = models.ForeignKey('api.GazetteerRegistryEntry',
                                null=True, blank=True, ...)

    stage = models.ForeignKey('Stage', ...) # editorial values ONLY
    owner = models.ForeignKey(settings.AUTH_USER_MODEL, ...)

    @property
    def is_prospect(self): # no Register row yet – we are still chasing it
        return self.registry_id is None
```

What that buys is considerable, because the Register already holds — and the ingest push already maintains — the licence (a resolved FK to `licensing.License`), rights holder, CRediT-shaped contributors, structured citation text, source URL, record count, temporal extent, spatial coverage and an owner. None of it needs re-entering, and none of it can drift, because there is only ever one copy. Every machine fact therefore disappears from GRACE's own fields, and what remains is genuinely ours to decide: the editorial stage, the responsible person, review status, permission to publish, when it came onto our radar, and the notes.

Pointing at the Register rather than at `datasets.Dataset` also makes a whole class of work expressible that would otherwise fall outside the system. Much of our outreach concerns *authorities* — the Getty over TGN, ISAW over Pleiades — not contributed datasets. A Dataset link cannot represent those conversations; a Register link can.

Why GRACE still needs its own table

The natural next thought is to put the editorial fields straight onto the Register row. There is even a precedent for it: the inventory push deliberately omits the curatorial fields (`core`, `no_explore`, `region_source`, `gazetteer_type`, `embargo_release_at`) from what it writes, with a comment warning that adding them would "silently reset them on every push, blowing away staff curation". Protect-by- omission is a proven, documented pattern here.

It still should not be done, for three reasons — the first decisive:

- **The primary key rules it out.** A Register row is keyed by namespace — `gn` for an authority, `whg:1234` for a WHG dataset — and that identifier exists only once something has been ingested. A print gazetteer someone mentioned at a conference has no namespace, so it can have no row without minting a placeholder key that would then have to change when the thing became real, breaking every foreign key pointing at it. Prospects are the majority of what GRACE tracks, so this is the main case, not an edge case.

- **status** is push-managed. It is set on every push, with a single carve-out protecting `embargoed`. The editorial stage can therefore never be `status`; it has to be a field of GRACE's own regardless.
- **The Register row is public-facing.** `/api/sources/`, the Atlas gazetteer inventory and the Regions offcanvas all read it through `visible_to()`. They use explicit field lists today, so nothing leaks by accident — but that discipline would have to hold across every future serializer once the same row also carried "permission unclear, chasing the archivist since March". That is a standing hazard for no benefit.

There is a lifecycle argument too: the Register row's existence is owned by the indexing side. A gazetteer we decline, or one later withdrawn, has no row — and its editorial history is exactly what we need to keep. Put plainly: **the Register is a gazetteer's published identity; GRACE is its editorial biography**, which begins earlier and outlives it.

Recommendation: give GRACE's gazetteer record a nullable FK to `GazetteerRegistryEntry`, keeping a local title so a prospect can be named before it has an identity. Read every machine fact through that link, and delete the derived values (*published*, *indexed*, *not indexed*) from the editorial stage list. A row with no Register link is a prospect, so no vocabulary has to say so.

Decision needed: confirm the editorial stage list once the derived values come out. That half is Palak's to define and needs no technical input.

3. Fix 2 — much of the Catalogue is already built

Two more of Palak's tables substantially exist in WHG already, and re-creating them would leave us with two competing copies of the same facts.

People is mostly `users.User`

The user model already carries `name`, `given_name`, `surname`, `affiliation`, `web_page`, `orcid`, `email`, `email_confirmed`, `role`, `is_active` and `preferred_language` — and `news_permitted`, which is precisely the mailing-list consent flag the draft's *Mailing lists* field implies. Palak's *Identifier* is `orcid`; her *active / dormant* is `is_active`.

The catch is that most people in the Catalogue will never have a WHG account — we have merely heard of them — and we must not create shell accounts for people we have only read about. So the right shape is a separate `Contact` model with an optional one-to-one link to `User`, and a firm rule that no field is stored in both places: where a Contact is linked, affiliation, ORCID and email are read *through* the link. This also turns Palak's own aside — "*ultimately the people submitting will become the user*" — from a note in the margin into a first-class relation.

Regions is already M49, and already built

Palak calls her region list a placeholder and asks for "a real scheme". WHG has one: `regions.Region` is a complete UN M49 hierarchy — `global / region / sub-region / intermediate / country` — with parent and member relations, ISO alpha-2 and alpha-3 codes, Wikidata identifiers, multilingual labels, and actual geometry and hulls. The placeholder can be dropped in favour of a many-to-many to `Region`, which also answers her question about multiple regions: yes, and it comes for free.

Recommendation: build `Contact` with an optional link to `User` and no duplicated fields, rather than a People table from scratch. Use `regions.Region` directly in place of the placeholder vocabulary.

4. Fix 3 — two entities are missing

Organisations

Affiliation is free text, and there is no institution record in the design — nor anywhere in WHG, so this genuinely is new. Permission to publish is granted by an archive, a library, a museum or a university, not by an individual, and *Permission to publish* is the field the entire licensing programme depends on. Projects are not institutions: a project ends, an institution does not, and the rights it granted outlive both the project and whoever we spoke to. A minimal Organisations model also gives us somewhere to record that we already hold a standing agreement with an institution before we approach a third researcher there.

Intake and triage

There is nowhere for a public submission to land. *Discovery source* offers `conference`; `referral`; `literature`; `flagged internally`; `other` and no value for a web submission, and nothing in the registers says "this arrived from the public and nobody has looked at it yet". The site's public suggestion form has to be built in Django anyway (§10), so the intake model and the form should be designed together rather than one retrofitted to the other.

Related: *Sources* → *Associated datasets* is documented as "datasets it documents". That is a **documentation** relation. There is no **derivation** relation — the candidate gazetteer that was extracted *from* a given print source. Since turning print gazetteers into WHG gazetteers is the main reason the bibliography exists, losing that link would cost us the provenance chain for the whole programme.

Recommendation: add an Organisations model; add `web form` to *Discovery source* and an *unreviewed* intake state; and split the Source → gazetteer link into "documents" and "derived from".

Decision needed: outstanding since June — is the public form a *suggest-a-source* tool, or a general front door that triages everyone and routes gazetteer owners on to the validate flow? The build of that form waits on the answer.

5. Fix 4 — preserving Palak's self-service

A no-code database lets Palak add a field or a vocabulary term herself, in seconds, without a developer. Django ordinarily does not, and self-service is the reason the team wanted a no-code tool in the first place. If we build in Django and ignore that, we will have solved the data-modelling problem while recreating the bottleneck it was meant to remove.

The mitigation is a modelling choice, and it needs making at the start rather than retrofitting: **model each controlled vocabulary as its own table, not as a `choices=` tuple**. A `choices` list is frozen in code and needs a developer and a migration to change; a lookup table is editable in the Django admin in ten seconds. Reserve `choices` for the handful of lists that code actually branches on — where adding a value without changing logic would be a bug — and make everything else a table Palak owns.

Applied to the draft's vocabulary list, that means *people roles*, *mailing lists*, *project status*, *source type*, *priority*, *contact status*, *interaction channel*, *action-item status*, *review recommendation*, *outcome*, *discovery source*, *digitization status*, *content type* and *content status* are all tables. Only the editorial stage is a plausible candidate for `choices`, and only if notifications branch on specific values.

Recommendation: vocabularies as tables by default, with an admin group giving Palak edit rights over them. Agree the short list of code-critical vocabularies that stay frozen, and say so in the model docstrings so nobody later "tidies" them into `choices`.

6. Field-level corrections

Field	As drafted	Change to	Why
Gazetteer · licence, rights, record count, coverage, reconciliation	Fields on the tracker	Read through the Register link	§2 — maintained by the ingest push; not ours to type
People · Name, affiliation, ORCID, email	Free text, re-declared	Read through the optional <code>User</code> link	§3 — never store the same fact twice
People · Mailing lists	Pick-list incl. <code>none</code>	Reconcile with <code>User.news_permitted</code>	Consent already has a home; two flags will disagree
Region (projects & gazetteers)	Single pick-list, placeholder values	M2M to <code>regions.Region</code>	M49 with geometry, ISO codes and labels is already built
Gazetteer · Time period	Free text, e.g. <code>900-1200</code>	Keep the prose, add numeric start/end year	Unsortable and unfilterable as text — and a prospect has no Register row to read it from
Gazetteer · Language(s)	Free text	ISO 639-3, multiple	The data is multilingual; free text will not group
Discovery source	No web-form value	Add <code>web form</code>	§4
Interactions · Logged by	Link to one of us	Drop	Duplicates the universal <i>added by</i> the draft says to assume on every record
Gazetteer · Status	Absent	Consider adding	People and Projects both carry one; the asymmetry is probably unintended

Three structural nits

- **There are four registers, not three.** Content is not engagement — it is output. The draft half-admits it (*"content ideas: quick jots — planned pieces live in Content"*). Either promote it, or drop the claim that each register is one working table; it is already untrue of the Catalogue, which is four.
- **Two Responsible person fields can disagree.** One sits on the Engagement, one on the tracked gazetteer, with no rule about which wins. Proposed rule: one accountable owner per gazetteer, inherited by its engagements unless deliberately overridden.
- **The notification rule misses the failure mode that matters.** "Stage change notifies the responsible person" is a good, restrained rule — but what goes wrong in outreach is *stalling*, and a stall is an *absence* of a stage change. Nothing fires when an engagement has sat in *awaiting reply* for six weeks. The fix is nearly free: make *Next follow-up* mandatory whenever the stage is an open one, and the existing date reminder becomes a staleness alarm. Worth suppressing the notification when the responsible person is the one who made the change — otherwise the system spends its credibility telling

people what they just did. All of this rides the existing `whgmail` machinery and Celery Beat rather than needing anything new.

7. The five questions Palak left open

Question	Answer
1. Regions — placeholder list; can something sit in more than one?	Already solved, and yes. <code>regions.Region</code> is a full UN M49 hierarchy with geometry, ISO codes, Wikidata ids and multilingual labels. Use a many-to-many to it and the multi-region question answers itself; single-region is the common case, not a constraint worth encoding.
2. Priority and stage — on the engagement, or on the catalog record?	The register logic answers it. A stage describing <i>the conversation</i> belongs on the Engagement; a stage describing <i>the thing</i> belongs on the thing. Contact status stays where it is (correct as drawn), and the editorial stage belongs to the tracked gazetteer. Priority is a property of the conversation: leave it on the Engagement.
3. Funding and grants — right home? Does it belong at all?	It belongs, for a reason the draft could not have known. Funder and grant number are part of the <i>credit chain</i> WHG already models — the Gazetteer Register carries CRediT contributors, rights holder and structured citation text. Keep <i>Funder</i> and <i>Grant number</i> on Projects, and note that they feed citation metadata. That is the justification, and it needs no new scope.
4. Dashboard stages — guesses, to be set with the technical director.	Half the question dissolves. The derived values are read through the Register link (§2). What remains is the genuinely editorial half, which is Palak's to define and needs no technical input at all.
5. External peer review — how much to build now?	None of it now — the draft is right to sketch and defer. Two notes for when we do: WHG's record-suggestion flow already runs <i>pending</i> / <i>approved</i> / <i>rejected</i> with a reviewer, timestamp and review note, and the Register's <i>embargoed</i> status already expresses "accepted but not yet public". Review comments could also reuse the existing comment model rather than inventing a parallel one.

8. On "dataset" and "gazetteer"

The draft uses "dataset" throughout, and it is worth being precise about why a find-and-replace would be the wrong correction. Palak's usage is not careless — it matches the current Django model name, which is exactly why it reads naturally.

The real issue is that her table holds *two different kinds of thing*: prospects (format `unknown`, permission `not yet asked`, someone mentioned it at a conference) and actual WHG gazetteers. "Dataset" is wrong for the second; "gazetteer" is premature for the first.

Recommendation: call the table **Gazetteers**, and let the nullable Register link from §2 carry the prospective-versus-actual distinction structurally. A row with a Register link is a gazetteer WHG holds; a row without one is a gazetteer we are still pursuing. No vocabulary has to express it, and nothing has to be reclassified by hand when a prospect lands.

Two riders. *Source type*: *gazetteer* stays as it is, but should be labelled "**printed gazetteer**" — that is the one place in the whole system where the word means a *lead* rather than a contribution, and the label should say so. And the code keeps the `Dataset` model name: this is a change to human-facing language, not a migration.

9. Naming the registers

Within GRACE, "Dashboard" has to go. WHG's dashboard is the logged-in contributor's own workspace — their gazetteers, their uploads — and this is a staff editorial queue with a different audience entirely. Both live in the same Django project, so the collision would be literal as well as conversational. Palak asks whether the two might merge; they should not, though her underlying instinct is sound and is answered by the `User` link in §3.

Recommendation: Catalogue · Engagement · Pipeline. "Pipeline" says what the third register is for — where a gazetteer sits on its way in — without borrowing a word that already means something else to our users.

10. Personal data

Holding this ourselves keeps the data at Pitt, under our own auth, backups and access control, with no third-party processor involved. But one detail makes the question pressing rather than routine.

WHG encrypts user emails at rest. `User.email` is an `EncryptedTextField`, with an indexed HMAC (`email_hash`) carrying equality lookups precisely because the column itself is unqueryable. A Contact table storing plain-text addresses for people who never signed up would hold *less*-protected personal data than we hold for people who did — a visible regression against the platform's own standard, in the same database. Whatever we build must meet the bar the user model already sets.

The rest follows from what the Catalogue is for. It will hold names, addresses, affiliations and mailing-list membership for researchers who never asked to be in a database. There is no recorded lawful basis, no consent capture beyond `news_permitted`, and no deletion path — *dormant* is a status, not erasure. Because contacts and account-holders sit in one system, a subject access request spans both, which is an argument for settling retention and erasure while the models are still on paper.

Decision needed: Ruth to confirm the lawful basis for holding contacts who never opted in, and the retention and erasure rule. Technical follow-on: match the encryption standard set by `users.User`.

11. Loose ends

An earlier attempt at this workflow ran on Baserow, an external no-code database, and left live code and content pointing at it. None of it is urgent except the first, which is a public front door.

Loose end	What to do
The public form <code>main/</code> <code>views.py::submit_dataset</code> , <code>/</code> <code>contribute/</code> , "Suggest a Source" in the Data menu	A 302 to an external Baserow form — so the site's public entry point for suggestions leads out of WHG entirely. Rebuild as a Django form writing to the intake model in §4; design the two together.
The licence sync <code>sync_licences_to_baserow</code>	Retire. Its purpose was to keep an external copy of the licence vocabulary in step, and the Register already resolves licences to <code>licensing.License</code> directly.

Settings and credentials <code>BASEROW_*</code> in <code>whg/settings.py</code>	Remove once nothing reads them. The bot credentials and API token should be <i>revoked at Baserow</i> , not merely unset — the token was circulated by email in the clear.
The bibliography	Held in a Baserow table, itself loaded from <code>WHG_gazetteer_bibliography.xlsx</code> . It needs exporting and importing into Sources — and it is the obvious first real content for GRACE.
The <code>leads</code> prototype	An earlier in-Django attempt — a lead model, an admin triage UI, a public form — sits on the <code>atlas</code> branch. Worth reading before starting: some of it is close to what §4 needs. Its recorded teardown SQL should not be run.
The Baserow design note <code>developer/baserow-workflow-tool.md</code>	Mark superseded rather than deleting — it records a decision and its reversal, which is worth keeping.

Summary — decisions to settle

Decision	The question	My recommendation
1. The Register link	Does GRACE point at <code>GazetteerRegistryEntry</code> , and read licence, rights, counts and coverage through it?	Yes — nullable FK, with a local title so prospects can be named.
2. Contacts vs users	One <code>Contact</code> model linked to <code>User</code> , or two separate people-tables?	One, with an optional link and no duplicated fields.
3. Vocabularies	Editable lookup tables, or frozen <code>choices</code> in code?	Tables by default — it is what preserves Palak's self-service.
4. Organisations	Do we add an institution entity, or keep affiliation as free text?	Add it — permission to publish is granted by institutions.
5. The public form (outstanding since June)	Suggest-a-source tool, or general front door? And who rebuilds it, when?	Suggest-a-source, with a visible untriated intake state.
6. Personal data	Lawful basis, consent, erasure path, and the encryption standard.	Settle before populating; match <code>users.User</code> .
7. Terminology and registers	Datasets → Gazetteers, and what the three registers are called.	Yes, with "printed gazetteer" for the print sense; Catalogue · Engagement · Pipeline.

Once the first four are settled the models can be written, and the build divides cleanly: the schema and the admin are a developer job, while the vocabularies, the views and the day-to-day shape of GRACE stay Palak's.